

■ Aora AI — Complete Technical Reference

System Architecture, RAG Deep Dive, Tech Stack & Comprehensive Interview Q&A; Guide

1. Tech Stack, Tools & Languages Breakdown

Aora AI is an interactive, production-ready AI learning SaaS platform that converts textbooks, PDFs, audio recordings, and YouTube videos into structured textbook-depth study guides, practice quizzes, and RAG-powered interactive workspace chat.

Category	Technologies Used	Why Used (Technical Rationale)
Frontend Framework	Next.js 16 (React 19, TypeScript)	Server-Side Rendering (SSR) for fast initial paint, optimized route prefetching, typed props for safety, and seamless Vercel deployment.
Frontend Styling & State	Tailwind CSS v4, Lucide Icons, Zustand	Utility-first glassmorphism design system. Zustand provides lightweight global state management for chat sessions and active notes without Context re-render overhead.
Authentication	Clerk Authentication	Enterprise-grade JWT session handling, social auth (Google/GitHub), and seamless frontend-backend security delegation.
Backend Web API	Python 3.10+, FastAPI, Uvicorn	Asynchronous ASGI execution framework with ultra-low latency, native async/await for IO operations (LLM APIs/Qdrant), Pydantic auto-validation, and OpenAPI docs.
Database & ORM	PostgreSQL (Neon DB), SQLAlchemy 2.0 (Asyncpg)	Relational persistence for users, notes, quizzes, and chat messages. Asyncpg driver handles non-blocking database queries under high concurrency.
Task Queue & Cache	Celery, Redis (Upstash Redis)	Offloads long-running CPU & network tasks (audio extraction, YouTube transcription, chunking, vector embedding) to background workers without blocking HTTP responses.
Vector Database	Qdrant Cloud (1536-dim vectors)	High-performance vector search engine supporting metadata payload filtering (`user_id`, `document_id`) for precise Retrieval-Augmented Generation (RAG).
AI Models & LLMs	Google Gemini 2.5 Flash, Groq Llama 3.3, OpenAI GPT-4o	Multi-provider fallback pipeline. Gemini Flash supports 1M token context for long video processing; Groq provides ultra-fast inference backup.

2. System Architecture & Execution Flow

The application follows an asynchronous decoupled architecture designed for high availability and low latency:

- **1. Ingestion Phase:** User uploads a PDF, DOCX, Audio file, or inputs a YouTube URL via Next.js frontend.
- **2. Async Queue Handoff:** FastAPI receives the request and immediately dispatches a task to Celery + Redis, returning a 202 Accepted response.
- **3. Text & Audio Extraction:** Background workers run pypdf, docx, youtube_transcript_api or fallback yt-dlp + Whisper/Gemini to extract raw text.

- **4. Chunking & Indexing:** Text is split into overlapping chunks (1000 chars, 200 char overlap) and upserted into Qdrant Vector DB with deterministic UUIDs.
- **5. LLM Synthesis:** Multi-provider LLM pipeline generates textbook-depth Markdown notes (2000-5000+ words) formatted with rich sections and cheat sheets.

3. How RAG (Retrieval-Augmented Generation) Works in Aora AI

Retrieval-Augmented Generation (RAG) grounds LLM responses in private user data (e.g. uploaded video transcripts or lecture notes) to eliminate hallucinations:

RAG Phase	Technical Implementation in Aora AI
1. Ingestion & Embedding	Raw document text is chunked into 1000-character blocks with 200-character overlaps. Each chunk is converted into a 1536-dimensional vector embedding and stored in Qdrant along with payload metadata (<code>`user_id`</code> , <code>`document_id`</code> , <code>`text`</code>).
2. Contextual Query Retrieval	When a user asks a question inside a document note, the query vector is searched against Qdrant. A payload filter (<code>`user_id == active_user` AND <code>`document_id == target_doc`</code></code>) isolates top-K cosine similarity chunks.
3. Prompt Augmentation	Retrieved context chunks are formatted into the LLM system prompt: Document Context: [Chunk 1, Chunk 2] \n\n User Question: This restricts the LLM to facts present in the text.
4. General vs Scoped Chat Handling	For global chat queries without a target document ID, RAG vector lookup is bypassed so the assistant answers general queries directly using native knowledge without false context errors.

4. Comprehensive Technical Interview Questions & Answers

Q1: Why did you choose FastAPI over Flask or Django for the backend?

FastAPI is natively built on asyncio and Starlette, making it 2 to 3 times faster than traditional synchronous frameworks like Flask or Django for IO-bound workloads (e.g. calling external Gemini/OpenAI APIs, PostgreSQL database queries, and vector DB searches). Additionally, FastAPI provides automatic Pydantic data validation and instant OpenAPI documentation generation.

Q2: Why do you use Celery and Redis for document and YouTube processing?

Extracting YouTube transcripts, processing long PDFs, or running audio downloads via ``yt-dlp`` can take anywhere from 15 to 90 seconds. If done inside a standard FastAPI HTTP route, it would block server worker threads and cause HTTP 504 gateway timeouts on client browsers. By delegating work to Celery workers backed by Upstash Redis, the API returns an instant HTTP 202 response while the background worker processes the document asynchronously.

Q3: How does RAG eliminate LLM hallucinations in your platform?

Standard LLMs rely purely on pre-trained parametric weights, which can hallucinate facts when asked about specific course materials. Our RAG architecture converts the student's actual document into vector embeddings stored in Qdrant. When a user asks a question, we query Qdrant for top matching text chunks and inject them directly into the LLM prompt context. This grounds the LLM strictly in factual source material.

Q4: How did you debug and resolve the issue where every YouTube video generated the exact same 'Data Structures' study note?

During root cause analysis of system logs, I discovered three chained issues: (1) ``youtube_transcript_api`` had an invalid method invocation (``list()`` instead of ``list_transcripts()``), causing transcript extraction to fail; (2) the fallback ``yt-dlp`` command had a 12-second timeout, which timed out on audio downloads; (3) when both failed, a fallback handler set ``doc_content`` to a hardcoded string containing 'data structures, algorithms'. The LLM then generated Data Structures notes for every video. I fixed this by correcting the transcript API call, increasing ``yt-dlp`` timeout to 120s with ``--no-playlist``, and replacing the hardcoded template with a dynamic topic framework derived from the video title.

Q5: How do you handle LLM rate limits and API failures in production?

We implemented a resilient multi-provider fallback pattern in ``llm_service.py``. The primary request targets Google Gemini 2.5 Flash. If Gemini throws a 429 Quota Exceeded error, the execution seamlessly fails over to Groq (Llama 3.3 70B), then OpenAI (GPT-4o), and finally a structured offline mock generator. This guarantees 99.9% application uptime for the user.

Q6: How are database transactions handled asynchronously in SQLAlchemy?

We use SQLAlchemy 2.0 with the ``asyncpg`` engine. Sessions are managed using ``async_sessionmaker`` and injected into FastAPI endpoints using dependency injection (``AsyncSession = Depends(deps.get_db)``). All DB interactions use ``await db.execute()`` and ``await db.commit()`` within explicit try/except blocks to ensure automatic rollback (``await db.rollback()``) on failure.

Q7: How do you secure API endpoints and manage authentication?

Authentication is powered by Clerk JWT tokens. The Next.js frontend sends an ``Authorization: Bearer `` header with every request. In FastAPI, a custom security dependency (``deps.get_current_user``) decodes the token claims and verifies the signature using Clerk's public JWT key, automatically syncing or fetching the user profile in PostgreSQL.

Q8: How is CORS handled across separate frontend (Vercel) and backend (Render) deployments?

CORS is configured in FastAPI using ``CORSMiddleware``. We explicitly whitelist production origins and configure dynamic regex matching (``allow_origin_regex=r'https://.*\.vercel\.app``). This allows Vercel preview builds and production domains to securely make authenticated cross-origin HTTP requests.